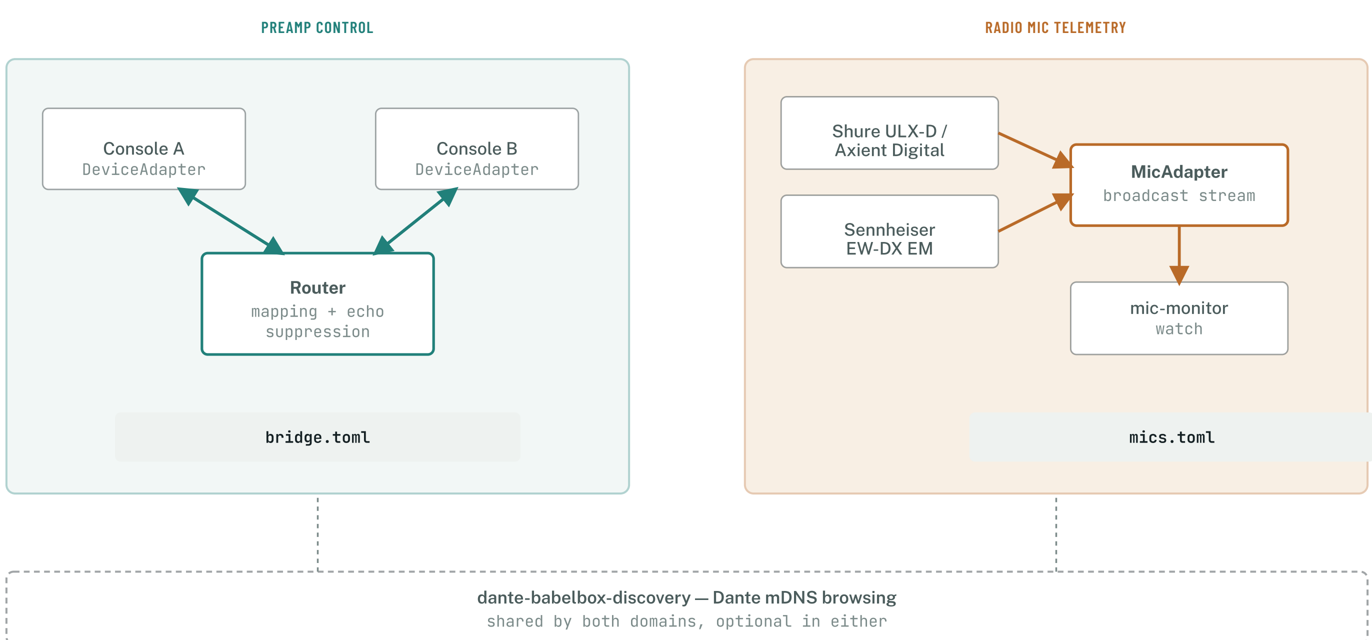
ARCHITECTURE GUIDE — RADIO MIC TELEMETRY

Watching Two Vendors Speak the Same Language

How Dante-BabelBox's newest domain connects to Shure and Sennheiser wireless mic receivers over their native control protocols, and surfaces battery, RF, and audio telemetry through one shared interface — without touching either vendor's audio path.

MICADAPTER TRAIT | NO HARDWARE REQUIRED TO TEST | DANTE-OPTIONAL | 45 TESTS, ALL MOCKED

DIAGRAM — TWO DOMAINS, ONE WORKSPACE 01



Same workspace, same discovery crate, two unrelated domains: preamp control still routes state between consoles; radio-mic telemetry only ever flows one way, into a terminal.

§1 Why telemetry gets its own trait

Preamp control is bidirectional and symmetric — a gain change has to propagate to a mapped peer, and confirmations have to be told apart from independent changes. Radio-mic telemetry isn't shaped like that at all.

DeviceAdapter → Router

`connect`, `set_gain`, `set_phantom`, `get_state`, `subscribe`. The `Router` holds a mapping table and fans state out to every mapped peer, tracking what it just pushed so a device's own confirmation doesn't bounce back and forth forever between two bidirectionally-mapped devices.

BUILT FOR TWO-WAY PROPAGATION

MicAdapter → broadcast stream

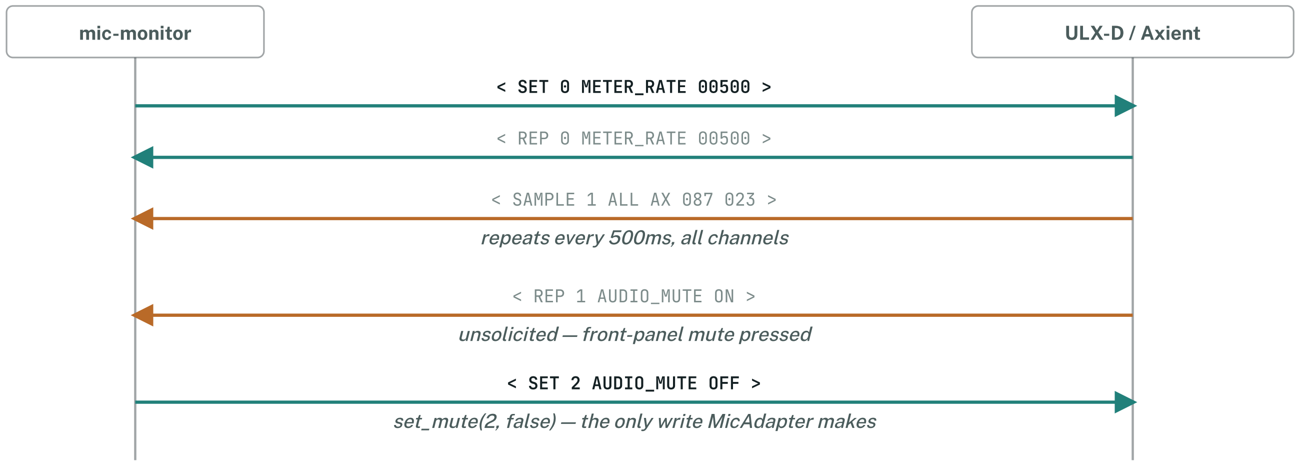
`connect`, `get_state`, `set_mute`, `subscribe`. Battery, RF, and audio level are monitoring-only; `mute` is the only realistic write. There's no peer to propagate to — telemetry just flows to whoever is subscribed.

BUILT FOR ONE-WAY MONITORING

§2 Shure — ASCII over TCP

ULX-D and Axient Digital share one plain-text protocol on TCP port 2202: four message types — GET, SET, REP, SAMPLE — framed with angle brackets.

SEQUENCE — CONNECT → METERING → UNSOLICITED UPDATES



One `SET 0 METER_RATE` at connect time turns on continuous metering for every channel — channel `0` means "all channels" for a dual/quad receiver.

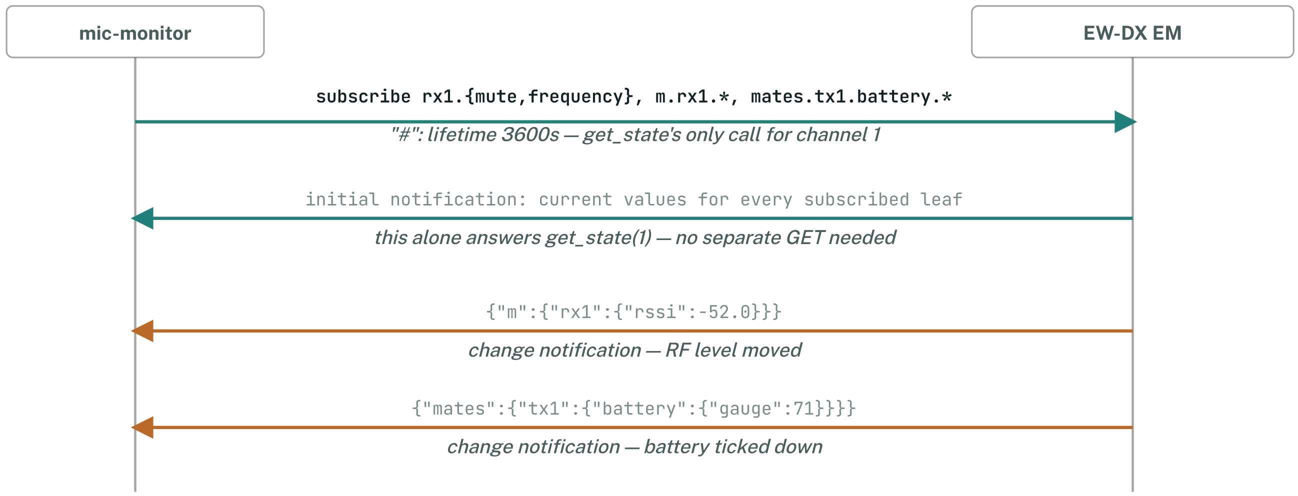
What Shure doesn't give you

No calibrated dBFS conversion for its raw audio meter (`SAMPLE`'s `eee`, 0–50), no distinct RF-quality percentage, and no device-identity query for the receiver itself. `MicState` leaves those fields `None` rather than guessing — see `mic-adapter-shure`'s module doc comment.

§3 Sennheiser — JSON over UDP

EW-DX speaks Sound Control Protocol (SSC) — JSON objects, one per UDP datagram, port 45 by default. Subscribing to a path delivers its current value immediately, then keeps pushing updates on change.

SEQUENCE — LAZY SUBSCRIBE PRIMES AND STREAMS IN ONE CALL



Subscriptions default to a 10-second lifetime; requesting `3600` up front avoids a renewal heartbeat for any reasonably-lengthed `watch` session.

the actual subscribe payload for channel 1

```
{
  "osc": { "state": { "subscribe": [{
    "#": { "lifetime": 3600 },
    "rx1": { "mute": null, "frequency": null },
    "m": { "rx1": { "rssi": null, "rsqi": null, "divi": null, "af": null } },
    "mates": { "tx1": { "battery": { "gauge": null, "lifetime": null } } }
  }] } }
}
```

Not the protocol this was first planned around

Research initially assumed EW-DX used the newer HTTPS+Server-Sent-Events "SSCv2" documented for other Sennheiser product lines. EW-DX's own developer guide states plainly it "supports only UDP/IP as transport protocol" — the plan adjusted before any code was written against the wrong assumption.

§4 What ends up in `MicState`

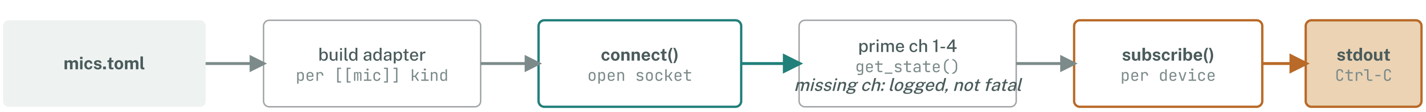
One shared struct, honestly populated per vendor — fields a protocol doesn't document stay None rather than being guessed at.

FIELD	SHURE ULX-D / AXIENT	SENNHEISER EW-DX
`battery_percent`	✓ **BATT_CHARGE**	✓ **battery/gauge**
`battery_minutes_remaining`	✓ **BATT_RUN_TIME**	✓ **battery/lifetime**
`rf_level_dbm`	✓ **SAMPLE aaa−128**	✓ **rssi (calibrated)**
`rf_quality_percent`	— no equivalent	✓ **rsqi**
`audio_level_dbfs`	— raw meter, no dBFS formula	✓ **af (genuine dBFS)**
`muted`	✓ **AUDIO_MUTE**	✓ **rx/mute**
`frequency_mhz`	✓ **FREQUENCY**	✓ **rx/frequency**
`antenna`	✓ **AX / XB / XX**	✓ **divi 0/1/2**

§5 What `mic-monitor watch` actually does

FLOW — CONFIG TO TERMINAL

—



One task per device merges its subscribe() stream into a shared stdout, tagged with device id and channel.

example output

```
ulxd-foh ch1: battery=82% runtime=300min rf=-45dBm quality=n/a af=n/a antenna=A
freq=614.125MHz mute=false
ewdx-1 ch1: battery=72% runtime=245min rf=-50dBm quality=88% af=-18dBFS antenna=A
freq=614.125MHz mute=false
```

A bug this diagram would have caught

The first working version dropped each adapter right after extracting its `subscribe()` receiver — dropping it closed the socket a few seconds later. Manual smoke-testing (not the unit tests, which mock the socket entirely) caught it before commit; the fix keeps every adapter alive for `run()`'s whole duration.

Part of Dante-BabelBox — see README.md and USAGE.md for the full status tables and config reference.

Not yet validated against real hardware — every test here runs against a mocked socket.